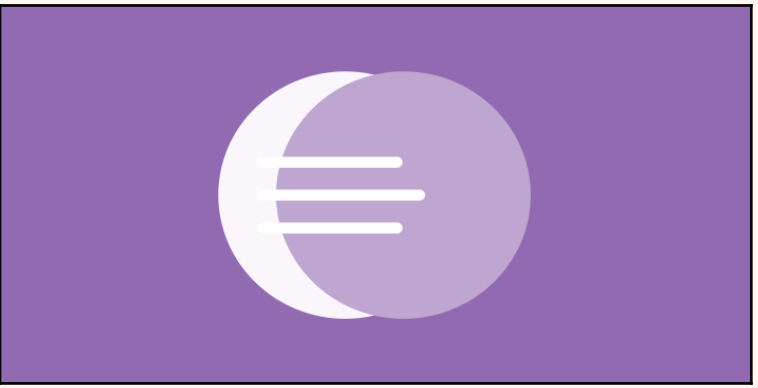
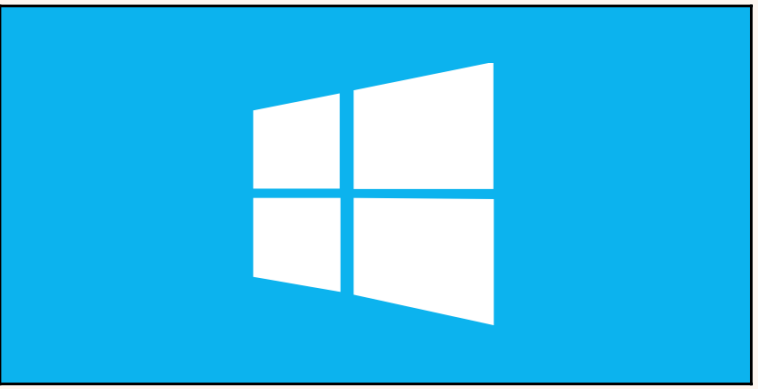






UD1: Fundamentos de Ingeniería del Software

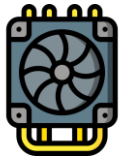
1º C.F.G.S. Desarrollo de Aplicaciones Multiplataforma
Módulo: Entornos de Desarrollo

Horas Previstas: 10h
RA Asociados: RA1

Índice

1. Software y Hardware del Ordenador
2. Lenguajes de Programación
3. Código Fuente, Código Objeto y Código Ejecutable
4. Máquinas Virtuales
5. Ingeniería del Software
    - 5.1. Fases del Desarrollo Software
    - 5.2. Roles Participantes
    - 5.3. Modelos del Ciclo de Vida
6. Metodologías Ágiles

1. Software y Hardware del Ordenador

Ordenador	 Hardware	Es la parte física del ordenador, formada por todos los componentes, tanto internos (CPU, memoria RAM, disco duro, placa base...) como externos o periféricos (teclado, ratón, monitor, impresora, etc.).						
								
	 Software	Es la parte lógica del ordenador, formada por los programas y aplicaciones que permiten controlar el hardware y realizar diferentes tareas, como navegar por Internet, escribir documentos o reproducir contenido multimedia.						
								

1. Software y Hardware del Ordenador

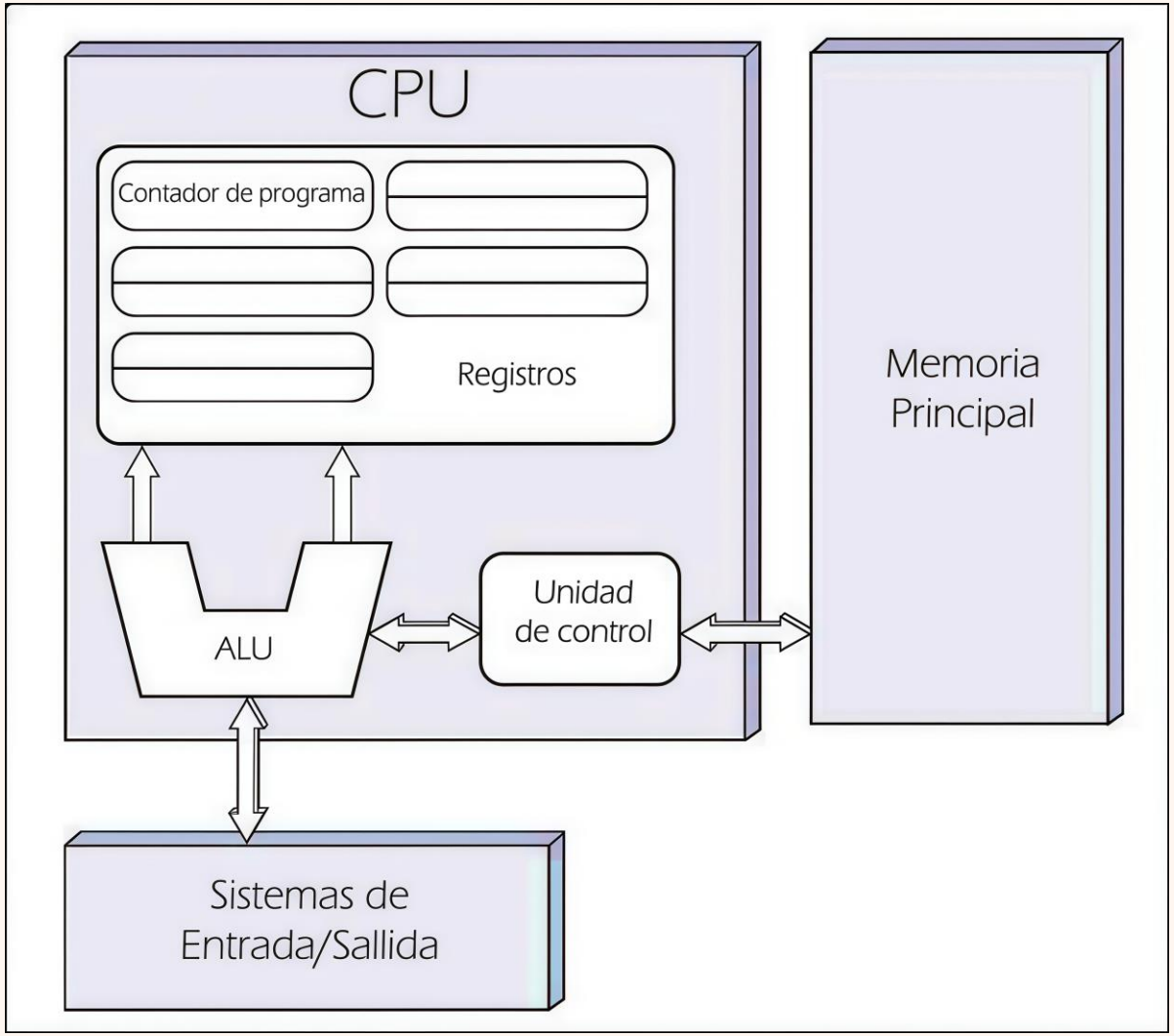
De acuerdo con la **arquitectura de Von Neumann**, un ordenador está compuesto por diversos componentes de hardware que trabajan conjuntamente para procesar datos y ejecutar instrucciones.

Unidad Central de Proceso (CPU)	
Es el componente encargado de **ejecutar las instrucciones de los programas**, traduciéndolas en operaciones aritméticas básicas (sumas, restas, …) y lógicas (OR, AND, …) sobre bits.	
Unidad Aritmético-Lógica (ALU)	**Ejecuta** las **operaciones matemáticas y lógicas** que le ordena la Unidad de Control y devuelve los resultados.
Unidad de Control (UC)	Obtiene las instrucciones de la memoria principal y dirige su ejecución e**nviando señales** a la **ALU** y a los **registros**.
Registros	Pequeño **almacenamiento interno de la CPU** que interviene durante la ejecución de las instrucciones (contador de programa, registro de instrucción, …).

Memoria Principal (Memoria RAM)
Almacena temporalmente las instrucciones y datos necesarios para que la CPU los procese. Es una memoria volátil, por lo que la información contenida en ella desaparece cuando el ordenador se apaga.

1. Software y Hardware del Ordenador

Unidad de Entrada/Salida	
Permite la **comunicación entre el ordenador y el exterior** mediante dispositivos llamados **periféricos**.	
Entrada	**Introducen datos** e instrucciones en el ordenador (teclado, ratón, escáner, micrófono, ...).
Salida	**Muestran al usuario los resultados** o la información procesada (monitor, impresora, altavoces, ...).
Entrada/ Salida	**Permiten la entrada y salida de datos**, así como el almacenamiento de información (tarjeta de red, discos duros, DVD, …).



1. El Software de un Ordenador (Definición)

El **software** de un ordenador se puede definir como el **conjunto de los programas** de cómputo, procedimientos, reglas, **documentación** y **datos asociados** que forman parte de las operaciones de un sistema de computación. (729 IEEE). En función de su uso se pueden clasificar en:

Software de Sistema	**Gestionan el hardware** y permiten el funcionamiento del ordenador, como sistemas operativos, controladores, ...				
Software de Aplicación	Diseñados para ayudar al usuario a **realizar tareas específicas**, como escribir documentos, navegar por Internet o jugar....				
Software de Programación	**Herramientas** utilizadas para **crear, probar y mantener programas**, como editores, compiladores y entornos de desarrollo (IDE).				

2. Lenguajes de Programación

Un **lenguaje de programación** es un **conjunto de símbolos y reglas que permiten escribir instrucciones** que un ordenador podrá ejecutar. Constará de los siguientes elementos:

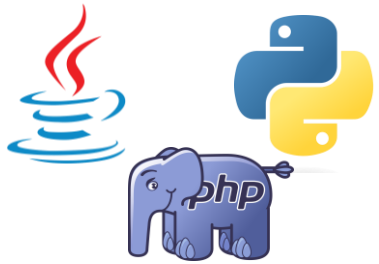
Alfabeto o Léxico	**Caracteres, símbolos, operadores y palabras reservadas** (if, while, return, …) que el lenguaje utiliza para construir instrucciones.	
Sintaxis	**Reglas** que establecen **cómo deben combinarse los símbolos** y las palabras para escribir instrucciones y estructuras de código válidas.	
Semántica	Significado de cada instrucción correctamente escrita, que determina la **acción que realizará el programa al ejecutarse**.	
Pragmática	**Uso práctico del lenguaje de programación** y su interacción con el compilador o intérprete, el sistema operativo y el entorno de ejecución.	

2. Lenguajes de Programación

<table>
<tr><td rowspan="4">Nivel de Abstracción</td><td rowspan="3">Lenguajes de Bajo Nivel</td><td colspan="2">Son los más próximos al funcionamiento interno del ordenador, ya que interactúan directamente con el hardware y las instrucciones que ejecuta el procesador.</td><td rowspan="3"><pre>
mov dx,OFFSET CADENA2
MOV AH,9H
INT 21H

mov ah,02h
    mov dl,10
    int 21h

    mov ah,02h
    mov dl,13
    int 21h

MOV AH,0AH
MOV DX,OFFSET CADENA
INT 21H
</pre></td></tr>
<tr><td>Lenguaje Máquina</td><td>Lenguaje Ensamblador</td></tr>
<tr><td>Usa solo 0 y 1 (binario) y es entendido directamente por el procesador, sin intermediarios.</td><td>Utiliza códigos nemotécnicos para representar las instrucciones y operar sobre los registros del procesador. Debe traducirse a lenguaje máquina.</td></tr>
<tr><td>Lenguajes de Nivel Medio</td><td colspan="2">Combinan características de los lenguajes de bajo y alto nivel. Se suele usar para la creación de sistemas operativos. Ejemplo: C.</td><td><pre>
int main()
{
  printf("Hola Mundo!\n");
  return 0;
}
</pre></td></tr>
<tr><td>Lenguajes de Alto Nivel</td><td colspan="2">Emplean una sintaxis cercana al lenguaje natural, lo que facilita su aprendizaje y uso. Son independientes del hardware y necesitan un compilador o intérprete para traducirse a lenguaje máquina. Ejemplos: Java, Python, C#, Pascal y COBOL.</td><td>  </td></tr>
</table>

2. Lenguajes de Programación

Paradigma de Programación (Lenguaje Alto Nivel)

Lenguajes Estructurados

Se centra en resolver un problema mediante una **secuencia ordenada de instrucciones**. Para ello, **utiliza** tres **estructuras de control básicas**: secuencial, condicional (alternativa) y repetitiva (bucles).

Las aplicaciones **se organizan en módulos**, cada uno encargado de realizar una tarea específica. Estos módulos **se comunican mediante llamadas**, pueden recibir parámetros (datos) y se clasifican en procedimientos, si no devuelven ningún valor, o funciones, si devuelven un resultado.

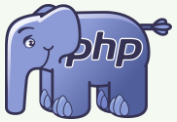







Lenguajes Orientados a Objetos

Supone un cambio de visión al **centrar el desarrollo en los datos** en lugar de los procesos. Las aplicaciones **se organizan en objetos**, que son instancias de una clase, y que interactúan entre sí. Cada objeto **posee atributos (datos)** y **métodos (acciones)**, y se comunica con otros objetos mediante llamadas a métodos.

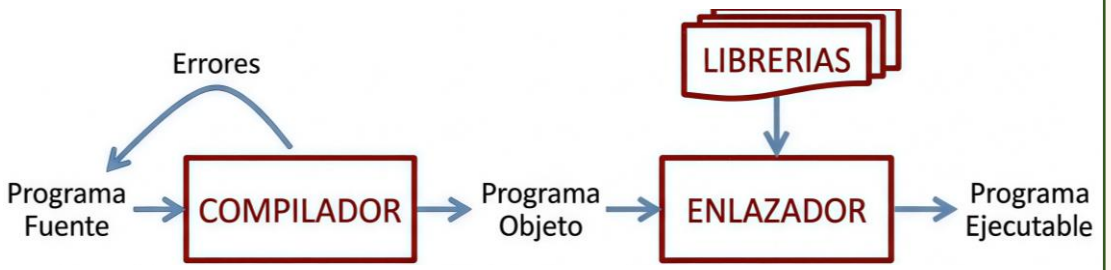
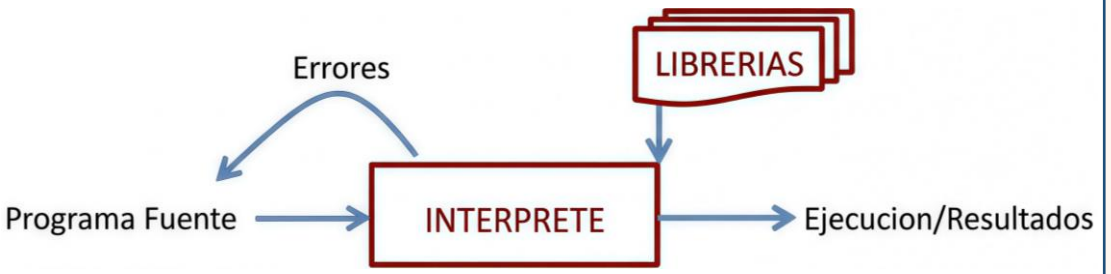








3. Código Fuente, Código Objeto y Código Ejecutable

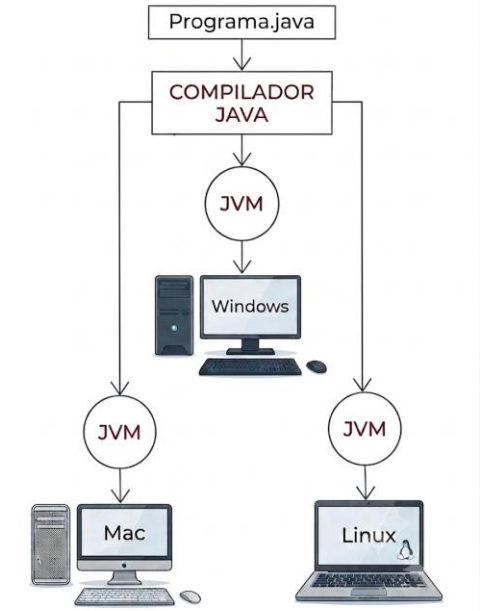
Para crear programas, los programadores **hacen uso de lenguajes de alto nivel** que no son entendidos directamente por los ordenadores (que utiliza código binario). Por este motivo, el **código fuente debe convertirse en código objeto** mediante un proceso de traducción o transformación, utilizando respectivamente:

Compilador	**Traducen todo el programa de una sola vez** para generar el código objeto. Luego, un **enlazador le añade las librerías necesarias** para crear el ejecutable final, reusable sin volver a compilar.	
Intérprete	**Traducen y ejecutan el código de manera simultánea** por fragmentos o bloques. Cada parte se almacena solo de forma provisional.	

Java usa tanto la compilación como la interpretación. Un programa fuente en Java se compila primero en un formato intermedio, llamado **bytecodes**, para que luego una máquina virtual lo interprete.

4. Máquinas Virtuales

Una máquina virtual **es una aplicación software capaz de ejecutar programas como si fuese una máquina real.** Se clasifican en dos categorías:

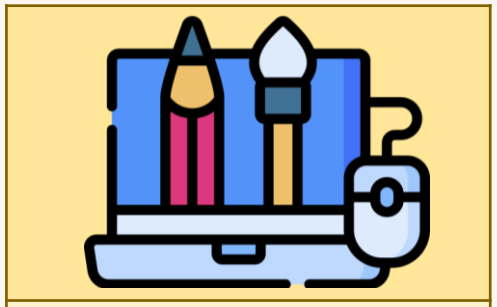
Máquinas Virtuales de Sistema	**Emulan un ordenador completo**, permitiendo instalar un SO diferente al del equipo físico. Son útiles para probar software, realizar prácticas o ejecutar varios sistemas operativos en un mismo ordenador.	
Máquinas Virtuales de Proceso	**Ejecutan una única aplicación de forma independiente** del sistema operativo, facilitando su portabilidad. Un ejemplo es la JVM, que ejecuta programas Java en cualquier plataforma compatible actuando como intermediario entre el código y el sistema. El **código fuente** (.java) **se compila a bytecode** (.class), y la JVM lo traduce a código máquina para que el procesador pueda ejecutarlo. 	

5. Ingeniería del Software

Es la disciplina que aplica **principios de ingeniería para diseñar, desarrollar y mantener aplicaciones informáticas** de manera eficiente, rentable y con alta calidad. Para ello, comprende las siguientes tareas:



Análisis



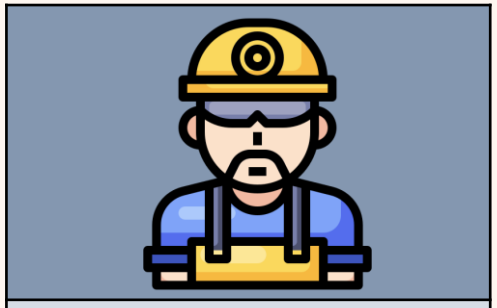
Diseño



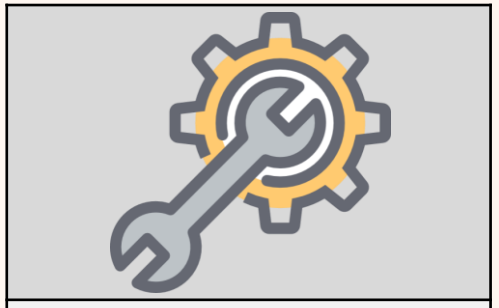
Codificación



Pruebas



Explotación



Mantenimiento

Cada etapa recibe y genera documentos de entrada y salida, convirtiendo a la **documentación en un pilar clave** del proceso.

5.1. Fases del Desarrollo Software (Análisis)

En esta fase se busca **comprender el problema y determinar los requisitos de la aplicación**, a partir de las necesidades del cliente. Los requisitos pueden ser de dos tipos:

Requisitos Funcionales	Requisitos No Funcionales
Indican las **funciones y servicios que debe ofrecer** la aplicación para satisfacer las necesidades del usuario.	Establece las **características de calidad** que debe cumplir la aplicación, como el rendimiento, la seguridad, la fiabilidad o la mantenibilidad.

Formas de Obtener Información			
	Entrevistas	**Reuniones** con el cliente o los usuarios **para recopilar información** sobre sus necesidades y requisitos.	
	Desarrollo Conjunto de Aplicaciones (JAD)	Sesiones de trabajo colaborativas en las que **usuarios y analistas definen y acuerdan conjuntamente los requisitos** de la aplicación.	
	Prototipos	**Versiones preliminares de la aplicación** que permiten validar y ajustar los requisitos antes del desarrollo.	

El resultado de esta fase es la **Especificación de Requisitos del Software (ERS)**, un documento que recoge los requisitos y sirve como base para el diseño de la aplicación.

5.1. Fases del Desarrollo Software (Diseño)

Una vez definidos los requisitos, se **diseña la solución que dará respuesta al problema**. En esta fase se establece la **estructura y el funcionamiento de la aplicación**, creando un modelo de software que servirá de guía para su implementación.

Diseño de Datos	Define cómo se organizarán y almacenarán los **datos del sistema** (tablas, archivos, variables y relaciones).	
Diseño Arquitectónico	Establece la **estructura general del sistema**, sus módulos, componentes y la forma en que se comunican entre sí.	
Diseño de la Interfaz	Diseña la **interacción entre el usuario y el sistema**, definiendo pantallas, menús y formularios.	
Diseño a Nivel de Componentes	Define el **funcionamiento interno de cada componente o módulo** del sistema, especificando su lógica, métodos o algoritmos mediante diagramas, pseudocódigo u otras técnicas de diseño.	

El **resultado de esta fase es la documentación de Diseño**, que puede incluir diagramas UML, modelos Entidad-Relación (E-R), diseño de interfaces, arquitectura del sistema y algoritmos (pseudocódigo o diagramas de flujo).

5.1. Fases del Desarrollo Software (Codificación)

Una vez diseñado el sistema, comienza la codificación, donde el diseño se convierte en código fuente. Para obtener un software de calidad, es importante seguir normas de programación. Algunas de ellas son:

<table>
<tr><td rowspan="3">Comentarios</td><td colspan="2">Comentarios de Documentación</td><td colspan="2">Comentarios de Implementación</td></tr>
<tr><td colspan="2">Documentan el código y permiten generar documentación con Javadoc.</td><td colspan="2">Explican el funcionamiento del código. Pueden ser de bloque o de línea.</td></tr>
<tr><td colspan="2"><pre>/**
 * Prueba es utilizada para ...
 */
public class Prueba {</pre></td><td><pre>/*
 * Comentario de bloque.
 /</pre></td><td><pre>/ Comentario de línea */

// Comentario corto</pre></td></tr>
<tr><td rowspan="5">Declaraciones y Nombres</td><td>Paquetes</td><td>Minúsculas. Organización jerárquica con puntos.</td><td><pre>java.io</pre></td></tr>
<tr><td>Clases e Interfaces</td><td>Nombres descriptivos en PascalCase.</td><td><pre>CalculadoraBasica
InterfazUsuario</pre></td></tr>
<tr><td>Métodos</td><td>Verbos en camelCase.</td><td><pre>calcularSuma(),
obtenerPromedio()</pre></td></tr>
<tr><td>Variables</td><td>Nombres cortos y descriptivos en camelCase.</td><td><pre>contador, numeroMaximo</pre></td></tr>
<tr><td>Constantes</td><td>Mayúsculas y guion bajo.</td><td><pre>PI
MAX_USUARIOS</pre></td></tr>
<tr><td>Indentación</td><td colspan="3">Líneas en blanco entre clases, métodos, ... y sangrados para facilitar la legibilidad del código.</td></tr>
</table>

Durante la codificación se **elaboran los manuales técnicos** (para desarrolladores) y se comenzara el manual de usuario.

5.1. Fases del Desarrollo Software (Pruebas)

En esta fase se **busca detectar errores**, no solo de codificación, sino también de especificación y diseño. Para ello se realizan diferentes pruebas **y se elabora el Informe de Resultados de Pruebas**.

Verificación	Comprueba que el software se construye correctamente.	
Validación	Verificar si el **producto es correcto y si se ajusta a los requisitos del cliente.**	

Las pruebas a realizar se considerarán **exitosas si se descubre un error no detectado** hasta entonces. Las recomendaciones para llevar a cabo las pruebas son las siguientes:

Evitar que el programador **pruebe sus propios programas**.
Incluir **todo tipos de datos**, tanto válidos/esperados como inválidos/inesperados.
Realizar únicamente **pruebas previamente planificadas** y documentadas.
Superar las pruebas **no supone la ausencia de errores.**
Realizar **pruebas desde el inicio** del desarrollo **reduce el coste** de corregir errores.
La mayoría de los defectos se concentran en pocos módulos



En el **diseño de pruebas** se puede utilizar el **enfoque de caja blanca** (centrado en la estructura interna del sistema) **y/o caja negra** (basado en los requisitos funcionales sin observar el funcionamiento interno del programa).

5.1. Fases del Desarrollo Software (Explotación)

Una vez probado el software, corregidos los errores detectados y documentado el proceso, comienza la fase de explotación. En esta etapa **se realiza la instalación, configuración y puesta en marcha del software** mediante las siguientes tareas:

Estrategia para la Implementación del Proceso	Se elabora un plan que **define normas y procedimientos** para ejecutar las actividades, gestionar incidencias y probar el software en el entorno de trabajo.	
Pruebas de Operación	Para cada **versión (release)**, se realizan **pruebas de funcionamiento**. Si los criterios son satisfactorios, se procede a **liberar el software**.	
Uso Operacional del Sistema	El sistema se pone en **funcionamiento en el entorno previsto**, siguiendo la **documentación del usuario.**	
Soporte al Usuario	Se brinda **asistencia técnica y consultoría**, registrando y supervisando todas las solicitudes y acciones realizadas.	

5.1. Fases del Desarrollo Software (Mantenimiento)

Durante la fase de mantenimiento, el software se actualiza para corregir errores, mejorar su funcionamiento o adaptarse a nuevos requisitos y entornos. Se distinguen cuatro tipos de mantenimiento:

Mantenimiento Adaptativo	**Adapta el software a cambios del entorno** (hardware, sistema operativo u otras aplicaciones). Es el más frecuente debido a la continua evolución tecnológica.	
Mantenimiento Correctivo	**Soluciona los fallos o errores detectados** durante el uso del programa para garantizar su correcto funcionamiento.	
Mantenimiento Perfectivo	**Añade mejoras y nuevas funcionalidades** a partir de las necesidades del usuario o para optimizar el rendimiento.	
Mantenimiento Preventivo	Facilita la **evolución futura del programa sin alterar sus funciones**, mejorando la comprensión del código, la mantenibilidad y la usabilidad.	

5.2. Roles Participantes

El desarrollo de software **requiere distintos roles**, que pueden ser desempeñados por una o varias personas según el proyecto, siendo alguno de estos:

Jefe de Proyecto
Planifica, gestiona, supervisa la ejecución y **coordina el éxito global del proyecto**.

Diseñador (Analista Técnico)
Diseña los componentes, módulos y la **solución técnica que servirá de guía** para la programación.

Expertos del Dominio
Conoce el negocio y colabora en la definición de las necesidades y requisitos del sistema.

Programador
Desarrolla y mantiene el código fuente siguiendo las especificaciones del diseño.

Analista (Analista Funcional)
Analiza las necesidades del cliente y las **determina los requisitos** que debe cumplir el sistema.

Probador (QA / Tester)
Realiza pruebas para detectar errores y verificar que el software cumple los requisitos de calidad.

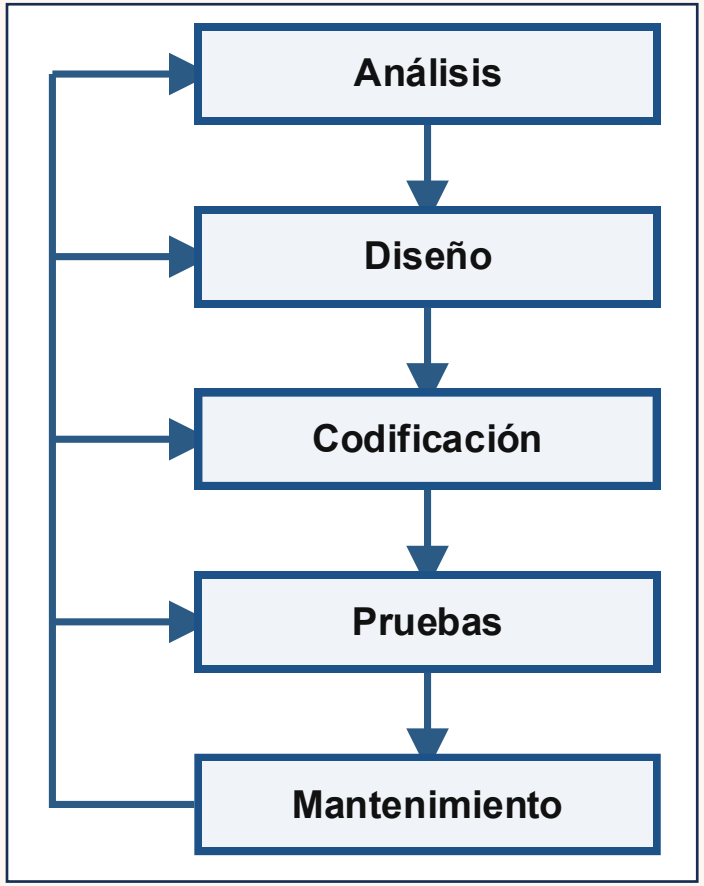
Arquitecto de Software
Define la **arquitectura, las tecnologías y la estructura** general que seguirá la aplicación.

Encargado de Implantación
Prepara, instala y **despliega la aplicación** en el entorno del cliente, gestionando las versiones.

5.3. Modelos del Ciclo de Vida (Cascada)

Los modelos determinan el orden de ejecución en el desarrollo de software. El pionero de estos enfoques fue el **modelo en cascada**, caracterizado por una estructura en la que **cada fase debe completarse** íntegramente para **dar inicio a la posterior.**

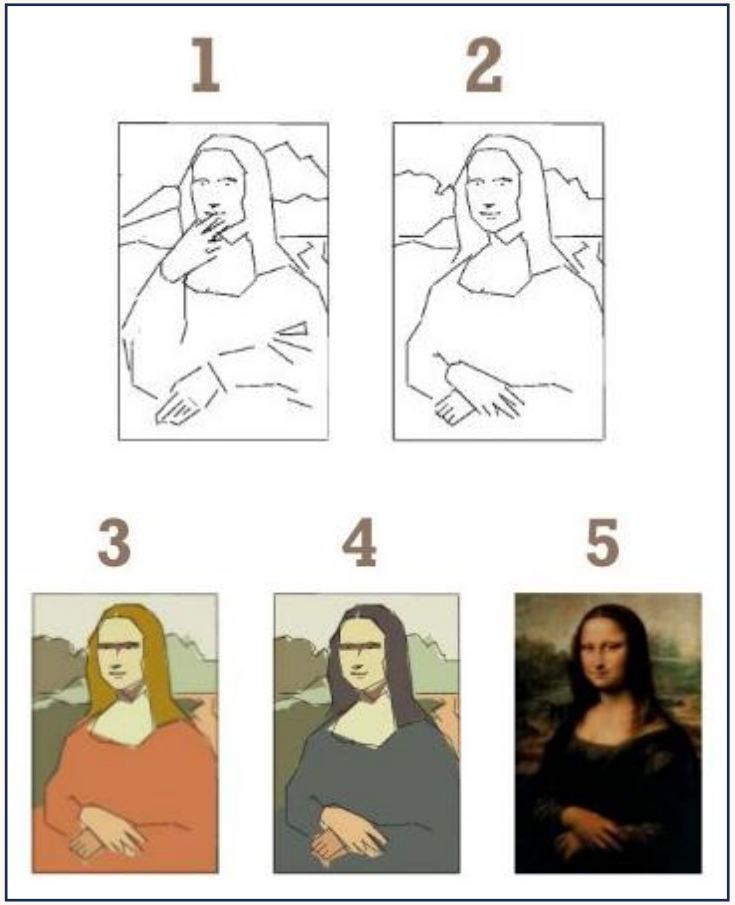
Ventaja	**Fácil de comprender**, planificar y seguir.
	La **calidad del producto resultante es alta.**
	Permite trabajar con **personal poco cualificado**.
Inconveniente	Necesidad de **establecer los requisitos** desde el principio.
	Es **difícil volver atrás** si se comente un error en alguna etapa.
	El producto solo **está disponible** una vez que **se ha completado** su desarrollo.
Recomendado	El **proyecto es similar a uno realizado** previamente con éxito, ya que se reduce la incertidumbre.
	Los **requisitos son estables** y están bien comprendidos.
	Los clientes **no necesitan versiones intermedias**.



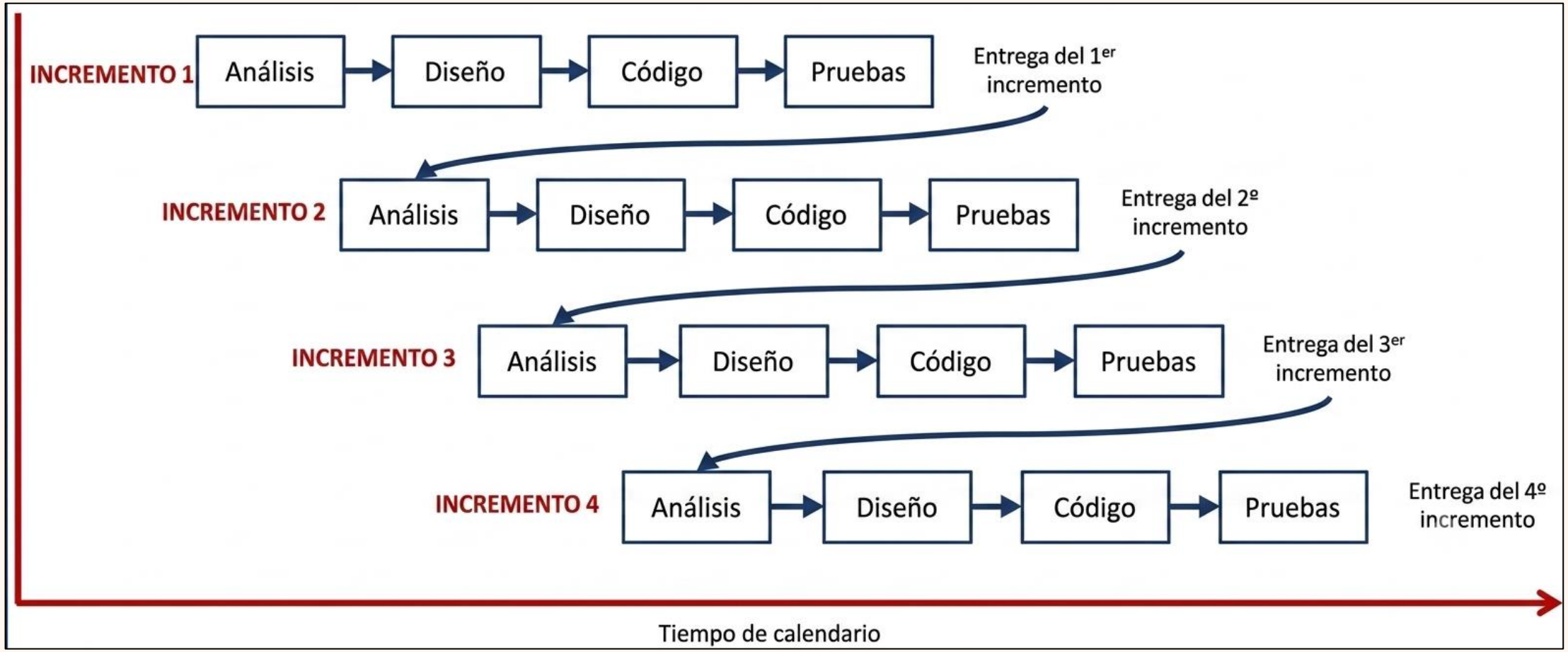
5.3. Modelos del Ciclo de Vida (Incremental)

Este modelo **evita esperar** al final del desarrollo **para disponer de una versión funcional**. Divide el proyecto en **varios ciclos de desarrollo**, **entregando** en cada uno un **incremento** funcional al que se añaden nuevas funcionalidades.

Ventaja	**No se necesitan conocer todos los requisitos** al comienzo.
	Permite la **entrega tempana** de partes operativas al cliente.
	La **retroalimentación** obtenida en cada entrega **facilita la mejora**.
Inconveniente	**Difícil estimar el esfuerzo** y el coste final necesario.
	Riesgo de no acabar nunca.
	No recomendable para sistemas en **tiempo real**, de **alta seguridad**, procesamiento distribuido o **alto riesgo.**
Recomendado	Los **requisitos** o el diseño **no están completamente definidos** y es posible que haya grandes cambios.
	El proyecto implica el **uso de tecnologías nuevas o poco conocidas**, por lo que es necesario probarlas y validar su funcionamiento durante el desarrollo.



5.3. Modelos del Ciclo de Vida (Incremental)

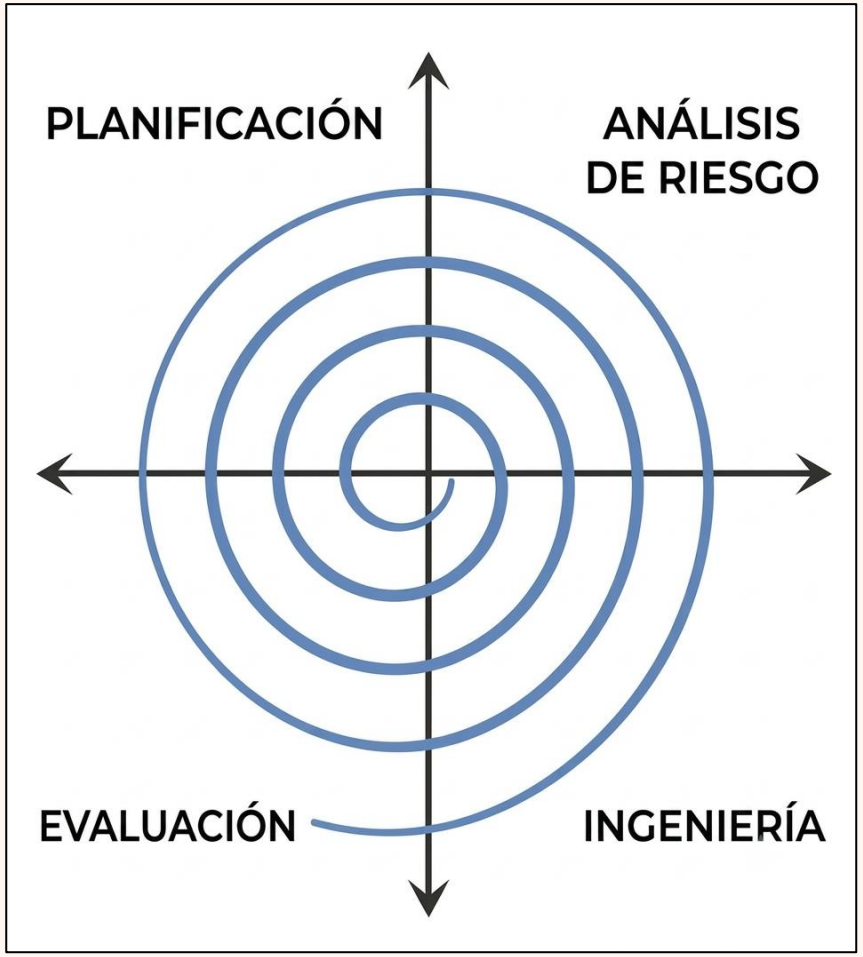


Cada ciclo produce una versión operativa del sistema, reduciendo el riesgo del proyecto y permitiendo validar los requisitos mediante la retroalimentación temprana del cliente.

5.3. Modelos del Ciclo de Vida (Espiral)

En este modelo, el desarrollo se representa como una espiral. En cada ciclo, formado por cuatro fases, se analizan los riesgos y se desarrolla una versión incremental del software.

Ventaja	No se necesitan definir todos los requisitos al comienzo.
	Análisis de riesgos en todas las etapas (reducción de riesgos).
	Incorpora objetivos de calidad.
Inconveniente	La identificación y evaluación de los riesgos puede resultar compleja y requiere experiencia.
	Los costes de desarrollo aumentan proporcionalmente con el avance de las vueltas en la espiral.
	El éxito del proyecto depende del análisis de los riesgos.
Recomendado	Proyectos de gran dimensión y complejidad que requieren una adaptación constante.
	Proyectos en los que la identificación, gestión y mitigación continua de los riesgos representan un factor crítico para el éxito del desarrollo.



5.3. Modelos del Ciclo de Vida (Espiral)

Cada ciclo dentro del modelo en espiral de desarrollo de software **se compone de cuatro tareas fundamentales**, desarrollan de forma secuencial

Planificación	**Define los objetivos del ciclo**, sus alternativas y las restricciones de tiempo y costes.	
Análisis de Riesgos	**Evalúa las alternativas** de desarrollo para seleccionar la más idónea y establecer medidas preventivas frente a posibles riesgos.	
Ingeniería	**Realiza el trabajo de análisis, diseño y programación** para construir una versión progresivamente más completa del sistema.	
Evaluación	El **cliente revisa el resultado obtenido** para decidir si el proyecto continúa hacia la siguiente iteración.	

En las **primeras iteraciones se emplean maquetas** para validar los requisitos, posteriormente se desarrollan prototipos funcionales parciales y, **finalmente**, se **entrega el sistema completo** e integrado.

6. Metodologías Ágiles

Tradicionalmente, el **desarrollo de software** seguía un **modelo secuencial**, como el modelo en cascada, con fases rígidas que **dificultaban la incorporación de cambios** durante el desarrollo y **retrasaban la validación** del producto hasta el final del proyecto.

Con el avance de la tecnología y la **necesidad de responder rápidamente a los cambios**, surgieron las **metodologías ágiles**. Sus principios quedaron recogidos en el *«Manifesto for Agile Software Development»* (2001), que establece los siguientes valores:

			
Individuos e interacciones sobre procesos y herramientas.	Software funcionando sobre documentación extensiva.	Colaboración con el cliente sobre negociación contractual.	Respuesta ante el cambio sobre seguir un plan.

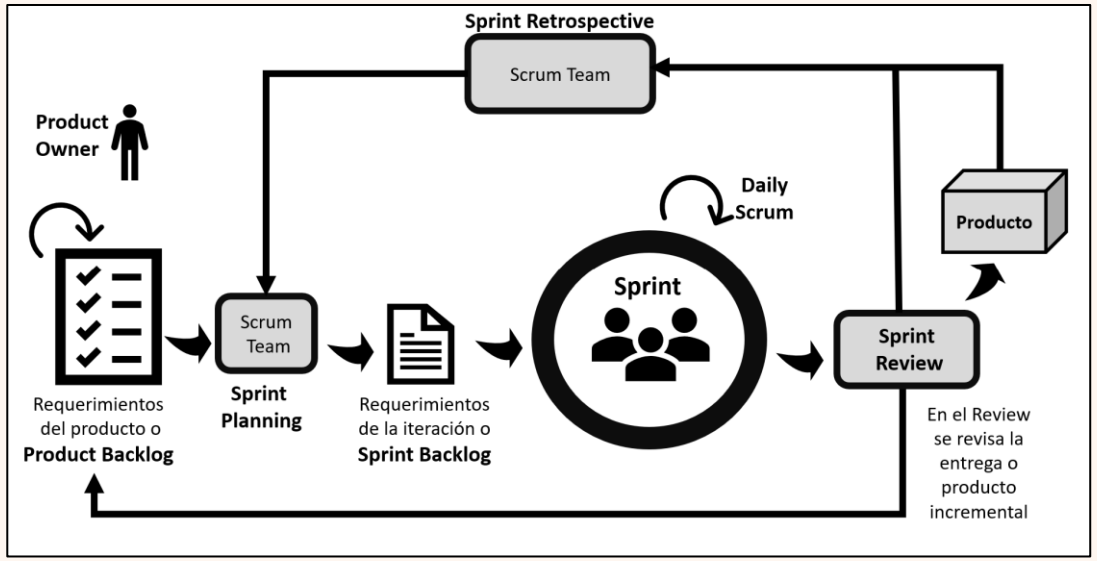
Las metodologías ágiles permiten **desarrollar software de forma iterativa e incremental**, entregando versiones funcionales de manera continua y facilitando la adaptación a los cambios.

9. Metodologías Ágiles (Scrum)

Busca **gestionar proyectos de forma colaborativa y flexible** mediante ciclos cortos llamados ***sprints***, entregando versiones funcionales del software y adaptándose a los cambios.

Product Owner	**Representa al cliente** y define prioridades para maximizar el valor del producto.
Scrum Master	**Líder del grupo de trabajo**, facilita el proceso, guía al equipo y elimina obstáculos.
Development Team	**Desarrolla el producto**, organiza su trabajo y entrega incrementos en cada Sprint.

Sprint Planning	**Reunión inicial** para seleccionar las funcionalidades del **Product Backlog** y crear el **Sprint Backlog** con las tareas del sprint.
Daily Scrum	**Reunión diaria** breve para revisar avances, coordinar el trabajo y detectar obstáculos.
Sprint Review	**Reunión final** para presentar el **incremento** y obtener **retroalimentación** que permita actualizar el **Product Backlog**.
Sprint Retrospective	Reunión del equipo para **analizar el sprint** e **identificar mejoras** para el siguiente.



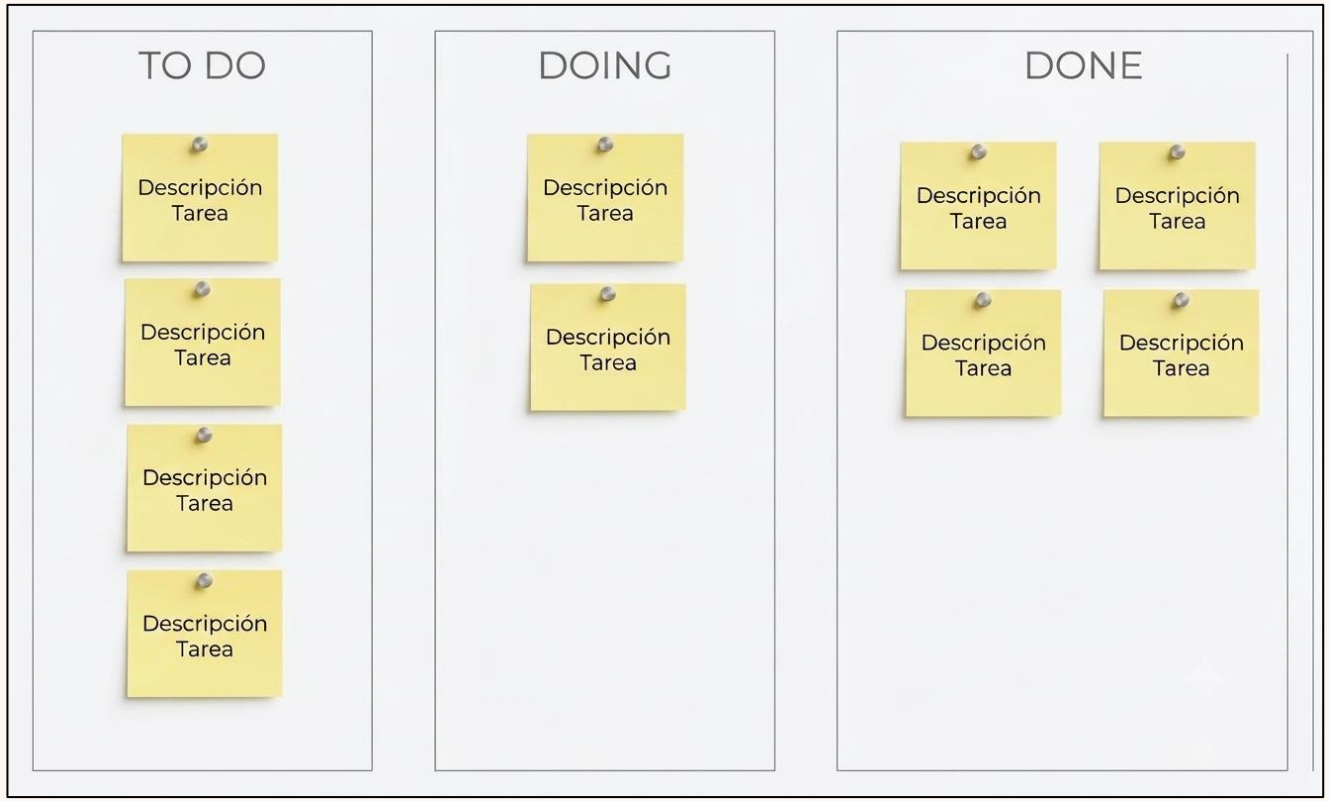
9. Metodologías Ágiles (Kamba)

Su origen esta en los procesos de producción “just-in-time” (JIT) de Toyota.

Este método **utiliza tarjetas visuales para mostrar el estado de cada tarea**, lo que permite ver de manera inmediata qué se está haciendo, qué está terminado y qué falta por hacer.

Esto facilita la colaboración dentro del equipo y ayuda a identificar posibles embotellamientos en el flujo de trabajo para poder corregirlos.

La metodología promueve un **enfoque iterativo**, empezando con un **producto mínimo viable** que funcione y, una vez aprobado por el cliente, se agregan progresivamente nuevas funciones.

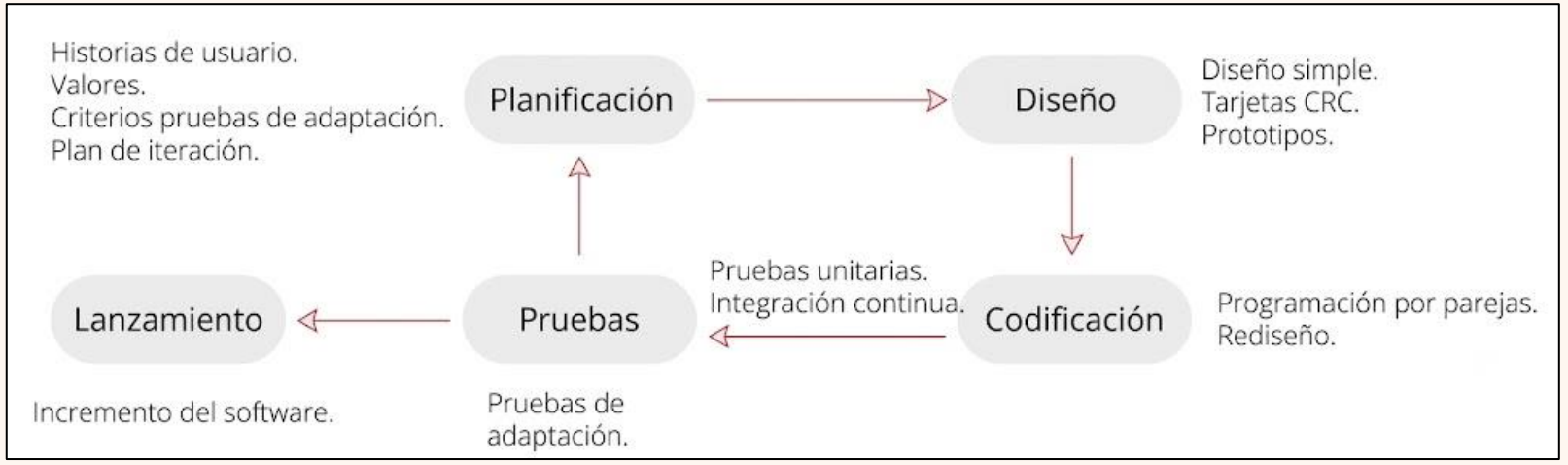


Algunas herramientas útiles para esta metodología son Trello, Asana o KambanTool.

9. Metodologías Ágiles (Extreme Programming XP)

Se basa en el **desarrollo iterativo** y en la **participación activa del cliente** durante todo el proyecto. Permite **adaptarse rápidamente** a los cambios en los requisitos mediante una planificación flexible y entregas frecuentes de software funcional. Se fundamenta en cinco valores:

Comunicación	Favorecer la **comunicación continua** entre el cliente y el equipo de desarrollo.
Simplicidad	Buscar desarrollar la **solución más sencilla** que satisfaga las necesidades.
Retroalimentación Rápida	**Entregar** software de **manera frecuente** para recibir comentarios y ajustar.
Coraje	**Cambiar y/o mejorar el código** cuando sea necesario.
Respeto	Todos los miembros del equipo **valoran y respetan las aportaciones** de los demás.



El desarrollo del software sigue las fases de planificación, diseño, codificación y pruebas. Cada **ciclo completo** (giro) **genera un incremento** del software que se entrega al cliente.